

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Shaik Azeem	Reg. No.:	192472307
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I Shaik Azeem(192472307)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shaik Azeem

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q49. Debugging Maximum–Minimum Finder Code (Comparison Branches Reversed)

The following code attempts to find both the minimum and maximum values in a list, but the update conditions are reversed, causing wrong outputs for many inputs. Carefully analyze each comparison branch and explain how the variables are being updated incorrectly. Apply systematic debugging to restore correct min-max tracking across the full array. Optimize the implementation for readability and correctness without changing the brute force nature of the scan. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```
def min_max(arr):
    mn = arr[0]
    mx = arr[0]
    for x in arr[1:]:
        if x > mn: # ERROR
            mn = x
        if x < mx: # ERROR
            mx = x
    return mn, mx
```

```
def min_max(arr):
    mn = arr[0]
    mx = arr[0]

    for x in arr[1:]:
        if x < mn:      # Update minimum
            mn = x
        if x > mx:      # Update maximum
            mx = x

    return mn, mx

a = [12, 5, 18, 3, 10]
print("Minimum and Maximum:", min_max(a))
```

```
PS C:\Users\shaik\Downloads\program\python\CSA0612> py test3.py
Minimum and Maximum: (3, 18)
```

Q50. Debugging Insertion Sort Code (Wrong Direction in Shift Comparison)

The following Insertion Sort implementation uses an incorrect comparison condition in the while loop, causing smaller elements to remain in the wrong position and breaking ascending order. Carefully analyze the shifting logic and explain why the sorted prefix is not maintained correctly. Debug the algorithm step-by-step so that larger elements shift right and the key is inserted in the proper place. Optimize the implementation for clarity and maintain correct indexing throughout. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean code with proper comments.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] < key: # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

```
1  def insertion_sort(arr):
2      for i in range(1, len(arr)):
3          key = arr[i]
4          j = i - 1
5
6          while j >= 0 and arr[j] > key: # Correct comparison
7              arr[j + 1] = arr[j]
8              j -= 1
9
10         arr[j + 1] = key
11
12     return arr
13
14     a = [12, 5, 18, 3, 10]
15     print("Sorted Array:", insertion_sort(a))
```

```
PS C:\Users\shaik\Downloads\program\python\CSA0612> py test4.py
Sorted Array: [3, 5, 10, 12, 18]
```